

DAY-3 — Linux Command Line

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **linux command line**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Linux Command Line is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: shells, command syntax, pipes, redirection, grep, find, sort, head/tail.

Reference	What to remember
<code>man <cmd></code>	read manual
<code>command --help</code>	quick help
<code>grep</code>	search text
<code>find</code>	search filesystem
<code>sort</code>	sort lines
<code>head/tail</code>	view beginning/end
<code> </code>	pipe output
<code>> / >></code>	redirect/append output

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. shells

A shell (like bash or zsh) is the program that reads the commands you type and executes them. It's also a small programming language in its own right, supporting variables, loops, and conditionals — which is why Day 7 builds directly on what you learn here. Knowing which shell you're using matters because syntax details (especially quoting) can differ between them.

2. command syntax

Most Linux commands follow the pattern `command [options] [arguments]` — for example `ls -la /home` runs `ls` with the `-la` options against the `/home` argument. Reading `man` or `command --help` is the reliable way to check exact syntax instead of guessing, especially before running anything with destructive potential like `rm` or `chmod`.

3. pipes

A pipe (`|`) takes the output of one command and feeds it directly as input to the next, letting you chain small tools into a powerful one-line workflow — e.g. `ps aux | grep ssh` lists processes and filters for "ssh" in one step. This composability is core to how security analysts work efficiently on the command line instead of manually scrolling through output.

4. redirection

Redirection sends output to a file instead of the screen: `>` overwrites the file, `>>` appends to it. This is how you save command output for evidence or later analysis — for example `nmap -oN out.txt` on Day 11 relies on exactly this concept, just built into the tool itself.

5. grep

`grep` searches text for lines matching a pattern, e.g. `grep -i failed /var/log/auth.log` finds failed login attempts, case-insensitively. It's arguably the single most-used tool in log analysis and incident response because it lets you pull the handful of relevant lines out of thousands instantly.

6. find

`find` searches the filesystem itself for files matching criteria — name, size, modification time, permissions — for example `find / -perm -4000` looks for files with the SUID bit set, a common privilege-escalation check. Unlike `grep`, which searches file contents, `find` searches file metadata and location.

7. sort

`sort` arranges lines of text in order (alphabetical, numeric, or reverse) and is often paired with other tools — e.g. sorting log entries by timestamp, or sorting `uniq -c` output to find the most frequent IP address hitting a web server.

8. head/tail

`head` shows the first lines of a file (default 10) and `tail` shows the last lines — very useful for quickly checking a large log file without opening the whole thing. `tail -f` follows a file live as new lines are appended, which is exactly how you'd watch a log in real time during an active investigation (see Day 19).

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
ls -la | head
find . -maxdepth 2 -type f
cat notes.txt | grep -i test
history | tail -n 10
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does shells solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain linux command line in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-3, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-3 — Linux Command Line

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `man` — read manual
- `command --help` — quick help
- `grep` — search text
- `find` — search filesystem
- `sort` — sort lines
- `head/tail` — view beginning/end
- `|` — pipe output
- `>` / `>>` — redirect/append output

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.